

SBA301

HƯỚNG DẪN HỌC TẬP VÀ PHỐI HỢP NHÓM

Integrate Single Page Application with Spring Boot

Syllabus ID	13801
Môn học	SBA301 - Integrate single page application with Spring Boot
Đối tượng	Sinh viên bậc đại học

Mục tiêu: giúp sinh viên biết cách tự học, học cùng nhóm, thực hành, nghiên cứu kỹ thuật và tích hợp kết quả vào project trong suốt học kỳ.

Tài liệu hướng dẫn triển khai học tập - phiên bản dành cho sinh viên

1. Cách sử dụng tài liệu này

Tài liệu này là “playbook” học tập cho toàn bộ môn SBA301. Sinh viên dùng nó để chuẩn bị bài, tổ chức nhóm, thực hành, nghiên cứu vấn đề kỹ thuật, quản lý project, lưu bằng chứng học tập và chuẩn bị cho các bài đánh giá. Đây không phải tài liệu thay thế slide, lab hay syllabus; nó hướng dẫn cách học hiệu quả với các tài liệu đó.

Nguyên tắc cốt lõi

Không học SBA301 theo cách “nghe giảng rồi chép code”. Mỗi kiến thức phải đi qua chu trình: chuẩn bị trước → giải thích cho người khác → thực hành → làm lỗi/kiểm thử → nghiên cứu → cải tiến project.

2. Căn cứ từ syllabus SBA301

SBA301 là môn 6 tín chỉ, tổng khối lượng 300 giờ, gồm 90 giờ contact, final exam 145 phút và khoảng 207,6 giờ self-study. Syllabus xác định phương pháp Learning-Teaching gồm Project-based Learning, Problem-based Learning, Active Learning và In-class Lecture. Vì vậy, phần lớn năng lực phải được hình thành thông qua tự học có hướng dẫn, hoạt động nhóm, thực hành và project chứ không chỉ qua giờ giảng.

Thành phần syllabus	Ý nghĩa đối với cách học
ReactJS, RESTful API, Spring Boot, Security, HTTP, Error/Exception Handling	Học theo chuỗi front-end → API → persistence → security → integration; không tách rời thành các mảnh kiến thức.
SQL Server, MongoDB, ReactJS tích hợp với Spring Boot	Phải thực hành trên hệ thống end-to-end, không chỉ làm từng snippet.
Teamwork và presentation	Mỗi thành viên phải tham gia giải thích, review và demo; không phân công kiểu “mỗi người biết một mảng duy nhất”.
Student tasks: đọc slide/tài liệu, làm exercise, self-practice	Chuẩn bị trước và tự thực hành là trách nhiệm bắt buộc, không phải phần bổ sung.
Tools: IntelliJ IDEA, JDK 21, Spring Boot 3, Node 21; SQL Server, MongoDB, ReactJS	Nhóm cần thống nhất môi trường và version ngay từ đầu để tránh lỗi tích hợp.

3. Kết quả học tập sinh viên cần hướng đến

Các CLO của syllabus có thể quy về bốn nhóm năng lực để sinh viên tự theo dõi tiến bộ:

Nhóm năng lực	CLO liên quan	Biểu hiện đạt yêu cầu

Nhóm năng lực	CLO liên quan	Biểu hiện đạt yêu cầu
Front-end & SPA	CLO1, CLO2	Xây được React component; quản lý props/state/hooks; routing; gọi API; xử lý dữ liệu phía client.
REST API & Persistence	CLO3, CLO4, CLO6, CLO7	Thiết kế và cài đặt REST API; kết nối JPA/Hibernate; làm việc với relational DB và MongoDB.
Security & quality	CLO5 + nội dung REST/testing	Thực hiện authentication/authorization; xử lý lỗi; kiểm thử, paging/sorting/versioning ở mức phù hợp.
Teamwork & communication	CLO8	Phân công, phối hợp, review, trình bày và bảo vệ quyết định kỹ thuật của nhóm.

4. Mô hình học P³R + Project Spine

Môn học được triển khai theo bốn hoạt động lặp lại, còn project đóng vai trò “xương sống” nối các chủ đề:

Thành phần	Sinh viên làm gì?	Sản phẩm
Prepare	Đọc slide/documentation, chạy ví dụ, ghi điều đã hiểu và điều chưa rõ.	Learning Note 3-2-1.
Peer	Giải thích cho bạn, thảo luận, phản biện, review code, đổi vai.	Giải thích miệng, câu hỏi, review comment.
Practice	Code, debug, test, sửa lỗi, tích hợp.	Code chạy + test/evidence.
Research	Đặt câu hỏi kỹ thuật, tìm evidence, làm experiment, kết luận.	Mini Research Note.
Project Spine	Đưa kiến thức mới vào một project phát triển dần trong học kỳ.	Phiên bản project theo milestone.

```

PREPARE → PEER → PRACTICE → RESEARCH → IMPROVE PROJECT
          ↑               ↓
        REVIEW ← TEST ← INTEGRATE
  
```

5. Chu trình học cho MỖI topic

5.1. Trước buổi học - Prepare

Mục tiêu của phần chuẩn bị là để sinh viên đến lớp với kiến thức nền và câu hỏi, không phải đến lớp để “nghe lần đầu”. Khuyến nghị dành 60-120 phút tùy độ khó của topic.

- Đọc slide và ít nhất một nguồn được chỉ định trong syllabus; ưu tiên official documentation.
- Chạy tối thiểu một ví dụ nhỏ. Không chỉ đọc code.

- Ghi Learning Note 3-2-1: 3 concepts, 2 experiments, 1 question.
- Đưa câu hỏi chưa rõ lên kênh nhóm trước buổi học để cả nhóm cùng chuẩn bị.
- Nếu dùng AI để giải thích/code mẫu, phải ghi lại điểm nào cần kiểm chứng bằng documentation hoặc experiment.

5.2. Trong buổi học - Explain, Build, Break

Pha	Hoạt động đề nghị	Vai trò sinh viên
1. Retrieval/Peer teaching (10-20 phút)	Câu hỏi nhanh, giải thích lại concept, peer discussion.	Trả lời bằng hiểu biết đã chuẩn bị; chỉ ra phần chưa chắc.
2. Lecturer clarification (20-30 phút)	Giảng viên tập trung misconception, architecture, trade-off, lỗi điển hình.	Ghi “điểm thay đổi nhận thức”, không chép lại slide.
3. Guided experiment (30-40 phút)	Chạy code mẫu, thay đổi input/configuration, quan sát hành vi.	Dự đoán → chạy → ghi kết quả → giải thích.
4. Challenge/Break it (30-40 phút)	Bài không có hướng dẫn chi tiết; hoặc code có lỗi cần debug.	Làm theo nhóm, tạo testcase và chứng minh fix.
5. Research seed (10-20 phút)	Một câu hỏi mở cho buổi sau.	Chốt câu hỏi, nguồn sẽ đọc và experiment cần làm.

5.3. Sau buổi học - Research, Integrate, Reflect

- Hoàn thành challenge/lab chưa xong và đảm bảo chương trình chạy lại từ đầu trên máy khác hoặc môi trường sạch.
- Thực hiện Mini Research Note nếu được giao hoặc nếu nhóm gặp một quyết định kỹ thuật chưa rõ.
- Đưa feature/bugfix vào project bằng issue → branch → commit → pull request → review → merge.
- Cập nhật Evidence Portfolio cá nhân: link commit/PR, screenshot, test, research note, điều đã học.
- Kết thúc bằng reflection 3-5 dòng: “Tôi hiểu thêm gì? Tôi còn yếu ở đâu? Tôi sẽ sửa thế nào trước buổi sau?”.

Quy tắc kiểm tra kiến thức

Một kiến thức chỉ được xem là “đã học” khi sinh viên có thể: (1) chạy nó, (2) giải thích nó, (3) sửa khi nó lỗi, và (4) áp dụng nó trong ngữ cảnh khác.

6. Tổ chức nhóm học tập và project

6.1. Quy mô và nguyên tắc

Khuyến nghị nhóm 4-5 sinh viên, duy trì xuyên suốt học kỳ. Nhóm hoạt động như một mini development team. Không chia cố định “một người React, một người backend, một người database” vì cách đó tạo dependency và làm thành viên không hiểu toàn hệ thống.

6.2. Vai trò luân phiên

Vai trò	Trách nhiệm chính	Bằng chứng
---------	-------------------	------------

Vai trò	Trách nhiệm chính	Bằng chứng
Topic Lead	Chuẩn bị topic, dẫn peer explanation, tổng hợp misconception.	Mini outline + câu hỏi thảo luận.
Developer	Cài đặt feature/prototype theo issue.	Commit/PR + chạy demo.
Tester / Debugger	Thiết kế testcase, tìm edge case, tái hiện lỗi.	Test log / bug report.
Reviewer	Review code/API/architecture, yêu cầu sửa trước merge.	Review comments + approval.
Researcher / Documenter	Tìm nguồn đáng tin, chạy experiment, ghi quyết định.	Research note / ADR ngắn.

Vai trò phải đổi theo milestone hoặc theo tuần. Mỗi sinh viên trong học kỳ cần có bằng chứng từng làm Developer, Reviewer/Tester và ít nhất một lần Topic Lead/Researcher.

6.3. Nhịp làm việc nhóm

Nhịp	Thời lượng	Nội dung
Pre-class sync	5-10 phút online trước buổi học	Ai đã đọc gì? Câu hỏi chung nào cần mang lên lớp?
Stand-up	5 phút khi bắt đầu buổi làm project	Đã làm gì? Sẽ làm gì? Blocker gì?
Review gate	Trước mỗi merge	Reviewer kiểm tra logic, naming, test, security, evidence.
Weekly retrospective	15-20 phút/tuần	Keep / Problem / Try: giữ gì, vấn đề gì, tuần tới thử gì?
Milestone demo	Sau mỗi learning mission	Demo feature, trả lời câu hỏi ngẫu nhiên, nhận feedback.

6.4. Quy tắc phối hợp bắt buộc

- Mọi việc có ý nghĩa phải có issue hoặc task rõ ràng; không giao việc bằng tin nhắn mơ hồ rồi mất dấu.
- Không commit trực tiếp feature lớn lên nhánh chính; dùng branch và pull request.
- Không merge code mà không có ít nhất một người khác review.
- Không chấm đóng góp bằng số lượng commit; đánh giá theo mức độ sở hữu vấn đề, chất lượng giải pháp, review/test và khả năng giải thích.
- Blocker kỹ thuật quá 30-45 phút phải được báo cho nhóm; sau khi nhóm thử, mới escalates cho giảng viên với evidence đã làm.
- Không để một thành viên giữ riêng database/configuration/secret khiến nhóm không thể chạy project.

7. Workflow project và Git



7.1. Definition of Ready (trước khi code)

- Issue có mục tiêu rõ và acceptance criteria tối thiểu.
- Biết API/data nào bị ảnh hưởng.
- Đã xác định người implement và người review.
- Nếu có quyết định mới/khó, đã đặt research question.

7.2. Definition of Done (trước khi đóng issue)

- Feature chạy được theo acceptance criteria.
- Có xử lý ít nhất happy path và error/boundary path chính.
- Có test hoặc evidence đủ để tái kiểm tra.
- Không có secret/password/token trong repo.
- Có PR/review và các comment quan trọng đã xử lý.
- Một thành viên khác ngoài người viết code có thể demo và giải thích feature.

8. Research-Based Learning dành cho môn framework

Research trong SBA301 không đồng nghĩa với viết paper. Mục tiêu là hình thành thói quen kỹ sư: đặt câu hỏi, tìm evidence, thực nghiệm và quyết định có căn cứ.

QUESTION → HYPOTHESIS → EVIDENCE → EXPERIMENT → OBSERVATION → CONCLUSION → PROJECT DECISION

8.1. Thứ tự ưu tiên nguồn

1. Primary sources: Spring / React / Hibernate / MongoDB official documentation và tài liệu chính thức được syllabus chỉ định.
2. Reliable secondary sources: sách, tài liệu kỹ thuật có uy tín, tutorial chuyên sâu được kiểm chứng.

3. AI/search: dùng để gợi ý, giải thích và mở rộng từ khóa; không dùng như nguồn cuối cùng khi kết luận kỹ thuật quan trọng.

8.2. Mẫu câu hỏi nghiên cứu

Chủ đề	Ví dụ câu hỏi
React	Khi nào nên lift state, dùng Context, hoặc dùng local state? Caching phía client giúp gì và có rủi ro stale data nào?
REST	PUT và PATCH khác nhau về semantics ra sao? API versioning bằng URL hay header có trade-off gì?
JPA/Hibernate	LAZY và EAGER ảnh hưởng truy vấn thế nào? N+1 query xuất hiện trong trường hợp nào? Entity có nên trả trực tiếp ra API?
Security	Authentication khác authorization thế nào? JWT và session khác nhau ở state, revoke và deployment?
Database	Khi nào MongoDB phù hợp hơn relational DB trong cùng một domain?
Architecture	Logic nào thuộc Controller, Service, Repository? Khi nào cần DTO/mapper?

8.3. Quy tắc mini experiment

- Chỉ thay đổi một yếu tố chính mỗi experiment khi có thể.
- Ghi rõ environment/version nếu kết quả có thể phụ thuộc framework.
- Quan sát bằng log, HTTP response, SQL query, test result hoặc đo lường cụ thể; tránh kết luận chỉ từ “cảm giác chạy được”.
- Kết luận phải nêu giới hạn: experiment này chứng minh điều gì và chưa chứng minh điều gì.

9. Sử dụng AI trong học tập

Nguyên tắc

AI may generate; student must verify. AI có thể hỗ trợ nhưng không thay thế trách nhiệm hiểu, kiểm chứng và bảo vệ code.

Được dùng AI cho	Sinh viên vẫn phải làm
Giải thích concept, gợi ý ví dụ, scaffold code	Đọc lại official docs và chạy code.
Tìm bug hoặc gợi ý nguyên nhân	Tái hiện lỗi, xác nhận nguyên nhân bằng evidence.
Sinh testcase	Kiểm tra coverage/edge case có thực sự phù hợp requirement.
Refactor/naming/documentation	Review diff và giải thích vì sao thay đổi tốt hơn.
So sánh hai giải pháp	Kiểm tra assumption, version và trade-off bằng nguồn/experiment.

Trong review/demo, giảng viên hoặc bạn học có thể chọn ngẫu nhiên một đoạn code và hỏi: “Nếu bỏ dòng này thì sao?”, “Request đi qua layer nào?”, “Vì sao chọn cách này?”. Nếu người sở hữu feature không trả lời được thì code đó chưa được xem là kiến thức đã làm chủ.

10. Lộ trình học xuyên suốt 120 session

Syllabus tổ chức 120 session với các chapter, lab, assignment, progress test, project assistant và project evaluation. Để dễ theo dõi, sinh viên gom chúng thành các Learning Mission dưới đây. Các mốc session là căn cứ từ syllabus; sản phẩm/evidence là hướng dẫn triển khai học tập.

Mission	Session	Trọng tâm syllabus	Đầu ra tối thiểu
M0 - Onboarding & Project Setup	1-3	Course introduction, environment, group project introduction	Môi trường thống nhất; nhóm; domain; repo; working agreement; project backlog v0.
M1 - React Foundation	4-18	React introduction, components, React-Bootstrap, props/state/context, hooks, Lab 01	React UI có component structure rõ; ít nhất 1 flow có state; Learning Notes và peer explanation.
M2 - SPA & Client-Server	19-34	Routes, client-server communication, fetching/caching, React Router/SPA, Lab 02, Assignment 01, Project Assistant 01, Progress Test 01	SPA có routing + API/mock data; chuẩn bị Assignment 01; project front-end skeleton.
M3 - REST API with Spring Boot	35-47	RESTful Web Services, documentation, versioning/paging/sorting/testing, Lab 03, Project Assistant 02	REST API chạy độc lập; OpenAPI/Swagger hoặc tài liệu endpoint; error handling/test evidence.
M4 - JPA & Full-stack Integration	48-71	JPA connection, Lab 04, Project Assistant 03, full SPA integration, Lab 05, CRUD React + REST, Assignment 02	CRUD end-to-end React <-> Spring Boot <-> relational DB; DTO/validation/error strategy có giải thích.
M5 - Security	72-81	Securing REST Services, Lab 06, Assignment 03	Protected API; authentication/authorization flow; negative tests 401/403; security research note.
M6 - Hibernate & Persistence Reasoning	82-96	Hibernate architecture/config/mapping; CRUD; Lab 07; Progress Test 03; Project Assistant 04	Mapping có căn cứ; theo dõi SQL; giải thích transaction/lazy/eager ở mức đã học; peer review persistence.
M7 - MongoDB & Spring Data	97-108	MongoDB, Spring Data MongoDB, NoSQL application, Lab 08, Project Assistant 05	Một module/use case NoSQL hoặc prototype so sánh; quyết định relational vs document có evidence.
M8 - DevOps & Final Project Evaluation	109-120	Spring Boot DevOps tools, Project Evaluation	Project chạy ổn định; README/run guide; evidence portfolio; demo chéo thành viên; retrospective cuối môn.

11. Quality Gate theo milestone

Gate	Câu hỏi bắt buộc trước khi qua gate
G0 - Environment	Tất cả thành viên clone và chạy được project? Version JDK/Node/DB thống nhất? Secret/config được quản lý an toàn?
G1 - Front-end	Component có trách nhiệm rõ? State/props/hook có thể giải thích? Routing chạy đúng?
G2 - API	Endpoint đúng HTTP semantics cơ bản? Response/error nhất quán? Có documentation/test?
G3 - Persistence	Entity/repository/service chạy đúng? Relationship/query không tạo lỗi rõ ràng? Có thể quan sát SQL?

Gate	Câu hỏi bắt buộc trước khi qua gate
G4 - Integration	React gọi được API thật? Validation/error hiển thị hợp lý? Có end-to-end demo?
G5 - Security	Anonymous/user/admin có quyền đúng? Negative test 401/403? Không lộ secret?
G6 - Final readiness	Máy khác chạy được theo README? Các thành viên demo chéo được? Feature quan trọng có evidence và owner?

12. Evidence Portfolio cá nhân

Mỗi sinh viên duy trì một portfolio nhỏ xuyên học kỳ. Mục tiêu không phải tăng thủ tục mà để chứng minh quá trình học và giúp chuẩn bị Q&A/final practical.

Evidence	Ví dụ
Learning Notes	3-2-1 của các topic khó hoặc được giao.
Implementation	Commit/PR có feature hoặc bugfix thực sự.
Review	Comment review có giá trị: phát hiện lỗi logic, security, API, test, naming hoặc design.
Testing/Debugging	Bug report, failing test, log trước/sau fix.
Research	Mini Research Note/ADR có evidence + experiment.
Communication	Topic lead note, milestone demo, retrospective contribution.
Reflection	Điểm yếu/điểm đã cải thiện và kế hoạch học kế tiếp.

Không dùng số commit làm thước đo duy nhất

Một commit nhỏ không đồng nghĩa đóng góp nhỏ; nhiều commit không đồng nghĩa hiểu sâu. Bằng chứng tốt phải cho thấy sinh viên sở hữu vấn đề, hiểu giải pháp, test/review và có thể giải thích.

13. Chuẩn bị cho các thành phần đánh giá

Syllabus quy định: Assignment 10%, Group Project 25%, Progress Test 10%, Final Exam 55%. Final gồm Practical Exam 25% (85 phút) và Theory Exam 30% (60 phút); mỗi phần final phải đạt ít nhất 4/10 và final result đạt tối thiểu 5/10 theo quy định trong syllabus.

Đánh giá	Cách học phù hợp	Sai lầm cần tránh
Assignment - 10%	Làm cá nhân có kiểm chứng; tự chạy từ đầu; có checklist concept + implementation.	Copy code chạy được nhưng không hiểu; chỉ luyện một dạng đề.
Group Project - 25%	Tích hợp dần theo mission; rotate role; evidence of contribution; demo chéo.	Một người làm phần lớn project; ghép module sát ngày demo.
Progress Test - 10%	Retrieval quiz hàng tuần; flashcards misconception; giải thích vì sao đáp án sai.	Học thuộc definition sát ngày test.
Practical Final - 25%	Luyện build/debug dưới timebox; khởi tạo project sạch; đọc error nhanh.	Phụ thuộc project cũ/template quá nhiều; không luyện thời gian.

Đánh giá	Cách học phù hợp	Sai lầm cần tránh
Theory Final - 30%	Concept map + scenario questions + so sánh trade-off; ôn xuyên kỳ.	Chỉ nhớ annotation/syntax; không hiểu request flow và architecture.

14. Kế hoạch tự học cá nhân

Do syllabus dành phần lớn thời lượng cho self-study, sinh viên nên lập lịch cố định. Một mẫu tối thiểu cho mỗi topic/tuần:

Thời điểm	Hoạt động	Kết quả phải có
Trước lớp	Đọc + chạy example + 3-2-1	Có ít nhất 1 câu hỏi thật.
Ngay sau lớp	Hoàn thành challenge/lab	Code chạy + ghi lỗi đã gặp.
Giữa tuần	Research hoặc project issue	1 PR/test/research note tùy nhiệm vụ.
Cuối tuần	Review + retrieval quiz + retrospective	Biết mình còn yếu CLO/topic nào.

Nếu một tuần quá tải, ưu tiên theo thứ tự: (1) kiến thức chuẩn bị cho topic kế tiếp, (2) hoàn thành lab/challenge, (3) project milestone, (4) research mở rộng. Không bỏ hoàn toàn retrieval/review vì đây là phần giúp giữ kiến thức đến final exam.

15. Quy trình xử lý khi “không biết làm”

1. Đọc error / requirement thật kỹ
2. Tái hiện lỗi nhỏ nhất
3. Kiểm tra documentation / version
4. Tìm trong project mẫu hoặc test tối thiểu
5. Hỏi bạn: nêu rõ đã thử gì
6. Dùng AI/search để gợi ý giả thuyết
7. Kiểm chứng giả thuyết bằng experiment
8. Nếu vẫn block: hỏi giảng viên kèm evidence

Khi hỏi hỗ trợ, không gửi câu “em bị lỗi, thầy xem giúp”. Hãy cung cấp: mục tiêu, error message, đoạn code tối thiểu, environment/version, điều đã thử, expected vs actual result. Đây là kỹ năng kỹ thuật quan trọng trong teamwork và nghề nghiệp.

16. Quy tắc lớp học và văn hóa nhóm

- Đến lớp với bài đã chuẩn bị; không để nhóm mất thời gian giải thích từ đầu những gì đã được giao đọc trước.
- Trình bày điều mình chưa hiểu một cách cụ thể; hỏi sớm hơn thay vì giấu đến sát deadline.
- Review code tập trung vào code và reasoning, không công kích người viết.
- Khi bất đồng kỹ thuật, chuyển tranh luận thành câu hỏi có thể kiểm chứng bằng documentation, test hoặc experiment.
- Luôn ghi nhận contribution của người review/test/research, không chỉ người viết code.

- Mỗi thành viên có trách nhiệm giúp nhóm có bus factor > 1: ít nhất một người khác phải biết chạy/giải thích phần mình làm.

17. Checklist tổng quát trước mỗi milestone

- ☐ Tất cả thành viên pull/clone và chạy được phiên bản hiện tại.
- ☐ Không còn feature quan trọng chỉ chạy trên máy một người.
- ☐ Issue quan trọng có acceptance criteria và owner.
- ☐ Pull request quan trọng đã được review.
- ☐ Có test/evidence cho happy path và error path chính.
- ☐ API/DB/security change đã được ghi trong README/documentation thích hợp.
- ☐ Không commit secret, token, password, dữ liệu nhạy cảm.
- ☐ Ít nhất hai thành viên có thể demo core workflow.
- ☐ Nhóm có thể giải thích 3 quyết định kỹ thuật quan trọng của milestone.
- ☐ Mỗi thành viên đã cập nhật evidence portfolio và reflection.

PHỤ LỤC A - Mẫu Learning Note 3-2-1

Mục	Sinh viên điền
Topic / Session	
3 concepts tôi hiểu	1) 2) 3)
2 experiments tôi đã chạy	1) 2)
1 question tôi chưa rõ	
Điểm cần verify	
Sau buổi học - điều tôi sửa lại trong hiểu biết	

PHỤ LỤC B - Mẫu Mini Research Note

Trường	Nội dung
Research Question	
Hypothesis	
Evidence / Sources	Official docs / book / technical source:
Experiment	Input / setup / version / code change:
Observation	Log / response / SQL / test result:
Conclusion	
Limitations	
Project Decision	Nhóm sẽ áp dụng / không áp dụng / thử thêm:

PHỤ LỤC C - Mẫu Issue / Task

Trường	Mẫu
Title	[Feature/Bug/Research] Mô tả ngắn, có động từ
Context	Vấn đề hoặc learning objective liên quan.
Acceptance Criteria	☐ ... ☐ ... ☐ ...
Technical Notes	API / component / entity / collection bị ảnh hưởng.
Owner / Reviewer	Owner: Reviewer:
Evidence required	PR / test / screenshot / research note / demo.

Trường	Mẫu
Definition of Done	Chạy + test + review + explain + documentation nếu cần.

PHỤ LỤC D - Checklist Pull Request / Code Review

- ☐ PR mô tả rõ mục tiêu và link issue.
- ☐ Code build/run; không có file rác hoặc secret.
- ☐ Naming và structure dễ hiểu; không lặp logic rõ ràng.
- ☐ Controller/Service/Repository có trách nhiệm hợp lý đối với phần đã học.
- ☐ Validation/error handling phù hợp với use case.
- ☐ API response/status code phù hợp ở mức kiến thức đã học.
- ☐ Persistence change không làm mất dữ liệu/mapping sai rõ ràng.
- ☐ Security-sensitive endpoint có kiểm tra quyền phù hợp.
- ☐ Có test/evidence cho thay đổi.
- ☐ Reviewer có thể giải thích feature sau khi review.

PHỤ LỤC E - Mẫu Weekly Retrospective

Mục	Gợi ý trả lời
KEEP - điều đang làm tốt	
PROBLEM - vấn đề/blocker	
TRY - thử nghiệm tuần tới	
Learning debt	Topic nào nhóm đang “chạy được nhưng chưa hiểu”?
Ownership risk	Phần nào hiện chỉ một người biết?
Next milestone	3 việc quan trọng nhất: 1) 2) 3)

PHỤ LỤC F - Mẫu Evidence Portfolio cá nhân

Ngày/Session	Topic/Issue	Vai trò	Evidence link	Tôi học được gì / còn thiếu gì

PHỤ LỤC G - Checklist chuẩn bị Project Evaluation

- ☐ README hướng dẫn setup/run đủ để máy mới chạy project.
- ☐ Có dữ liệu mẫu hoặc script/seed cần thiết.
- ☐ Core workflow hoạt động end-to-end.
- ☐ Nhóm có kịch bản demo chính và kịch bản fallback khi một service lỗi.
- ☐ Mỗi thành viên chuẩn bị giải thích architecture/request flow.
- ☐ Mỗi thành viên có thể demo ít nhất một phần không phải do mình code chính.
- ☐ Danh sách known issues/limitations được ghi rõ; không che lỗi đã biết.
- ☐ Các quyết định security/persistence quan trọng có evidence/reasoning.
- ☐ Nhóm đã rehearsal Q&A: “tại sao?”, “nếu đổi X?”, “lỗi Y xảy ra thì sao?”.
- ☐ Evidence portfolio cá nhân được cập nhật trước evaluation.

18. Thông điệp cuối cùng cho sinh viên

Mục tiêu thực sự của SBA301

Không phải nhớ toàn bộ Spring Boot hoặc React. Mục tiêu là hình thành khả năng học một framework mới: đọc tài liệu, thử nghiệm, giải thích, debug, làm việc với người khác, đưa ra quyết định có căn cứ và xây được hệ thống chạy được.

Hãy duy trì nhịp học đều từ đầu kỳ. Project không phải “một bài lớn làm ở cuối môn”; project là nơi kiến thức của từng chapter được thử nghiệm và kết nối. Nhóm không chỉ chia việc để hoàn thành sản phẩm; nhóm là môi trường để mỗi thành viên học sâu hơn thông qua giải thích, review và phản biện.